Technical Explanation — NexEra AI Prototypes

What I Built

Two interactive AI-powered prototypes deployed as a single Next.js web application:

1. **AI-Generated 3D Asset Pipeline** — Takes text descriptions or images, generates 3D models via Tripo3D, displays them in an interactive viewer, and provides AI-generated educational summaries via Gemini (including Vision for image uploads).
2. **Natural Language Avatar Animation** — A procedural 3D avatar that interprets plain English commands using Gemini, then performs spatially-aware animations — walking to objects, pointing at targets, turning around — with AI explanations for each action.

Why I Chose This Approach

Next.js + React Three Fiber: Next.js provides both the React frontend and secure server-side API routes in a single deployment, keeping API keys safe while enabling one-click Vercel hosting. React Three Fiber brings Three.js into React's declarative model, making 3D scenes composable and maintainable.

Tripo3D: Produces GLB models directly from text or images via a single API, with clean web-ready topology. Models are proxied through a Next.js API route to eliminate CORS issues entirely.

Google Gemini (Multi-Model Fallback): Handles educational summaries (text + vision) and NLP command parsing. A fallback system tries multiple Gemini models in sequence, recovering automatically from rate limits or deprecations.

Procedural Avatar: Rather than relying on external animation assets, I built the avatar from Three.js primitives with a custom animation system — demonstrating deeper engineering while shipping as pure code with zero asset dependencies.

Architecture and AI Logic

Test 1 follows a create → poll → display pipeline: user input hits a Next.js API route that creates a Tripo3D task, the client polls for completion, and the finished GLB is proxied server-side and rendered in a React Three Fiber viewer with auto-scaling and centering. Gemini generates the educational summary in parallel.

Test 2 sends natural language to Gemini along with scene context (available animations, object names and 3D positions). Gemini returns structured JSON: `{ animation, target,`

`targetPosition, standoff, explanation }`. The avatar then executes spatially — calculating a destination point offset by the standoff distance, rotating to face the target, walking there, and performing the action. Animations blend smoothly via snapshot-and-interpolation between poses.

Challenges and Solutions

CORS with 3D Models: Tripo3D's CDN lacks CORS headers. I created a `/api/proxy-model` endpoint that fetches GLB files server-side and streams them to the browser — clean, platform-agnostic, and invisible to the user.

Animation Blending Without a Skeleton: Without pre-made animation files, I built a system that captures full joint state as a snapshot before transitions, then interpolates to the new animation's output over ~0.3s for smooth crossfades.

Spatial Navigation: Moving an avatar to an object requires path calculation, rotation, movement, and stopping at the right distance. I implemented world-space position/rotation tracking with per-object standoff distances to prevent the avatar from walking through objects.

Consistent Model Display: Generated models arrive at wildly different scales. The viewer computes each model's bounding box, normalizes to a target size, and positions it on the ground plane with automatic camera reset.

How I Would Scale This Inside NexEra's Platform

Short Term: Replace the procedural avatar with ReadyPlayerMe/Mixamo characters, expand to 50+ animations, add persistent storage for generated 3D asset libraries, and implement per-organization authentication.

Medium Term: Build a scenario editor for multi-step training sequences, add speech-to-text for voice-controlled avatars, implement multi-avatar scenes, and create an assessment engine that tracks learner competency.

Long Term: Fine-tune domain-specific models, integrate with LMS platforms (SCORM/xAPI), deploy the avatar system as a reusable SDK, and add real-time multiplayer for collaborative training.